

Введение в NoSQL и работа с документо-ориентированной базой данных MongoDB

Цель

Целью лабораторной работы является освоение практических навыков работы с документо-ориентированной базой данных MongoDB. Будет изучена разница между NoSQL и реляционными базами данных, а также выполнены базовые и сложные операции в MongoDB.

Работа с MongoDB ведется при помощи Docker — платформы, предназначенной для разработки, развёртывания и запуска приложений в контейнерах.

Содержание

[Введение в NoSQL и классификация баз данных](#)

[Документо-ориентированные базы данных на примере MongoDB](#)

[Самостоятельная работа](#)

[Проверка выполненного отчета](#)

[Дополнительные материалы](#)

Введение в NoSQL и классификация баз данных {#введение-в-posql-и-классификация-баз-данных}

Понятие NoSQL

NoSQL (*Not Only SQL*) — это класс систем управления базами данных, которые отличаются от реляционных СУБД и предназначены для хранения и обработки больших объемов неструктурированных данных. NoSQL базы данных обеспечивают гибкость модели данных, высокую производительность, масштабируемость и устойчивость к отказам.

Отличия NoSQL от реляционных баз данных

Отсутствие жесткой схемы данных

NoSQL базы данных не требуют строгой структуры таблиц и типов данных, как это необходимо в реляционных СУБД.

Горизонтальное масштабирование

Большинство NoSQL решений обеспечивают масштабируемость за счет добавления новых серверов, а не повышения мощности одного узла.

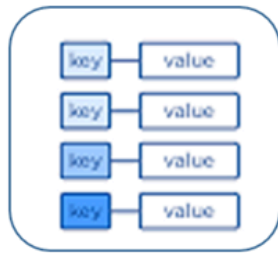
Производительность и оптимизация для конкретных задач

NoSQL базы данных оптимизированы для специфических сценариев использования, таких как хранение больших объемов текстовых данных или графов.

Классификация NoSQL баз данных



Document
Store



Key-Value
Store



Wide-Column
Store



Graph
Store

 [Источник](#) изображения

Документо-ориентированные базы данных (Document-based)

Хранят данные в виде документов (например, JSON или BSON), что позволяет гибко управлять структурой записей (MongoDB, CouchDB).

Колоночные базы данных (Column-family)

Хранят данные в виде столбцов и оптимизированы для выполнения аналитических запросов (Cassandra, HBase).

Графовые базы данных (Graph-based)

Предназначены для хранения данных в виде графов, что эффективно для поиска взаимосвязей и работы с социальными сетями (Neo4j, OrientDB).

Базы данных типа ключ-значение (Key-Value)

Хранят пары ключ-значение, оптимизированы для быстрого доступа по ключу (Redis, DynamoDB).

Docker

Предшественниками контейнеров Docker были виртуальные машины. Виртуальная машина, как и контейнер, изолирует от внешней среды приложение и его зависимости. Однако контейнеры Docker обладают преимуществами перед виртуальными машинами: потребляют меньше ресурсов, их очень легко переносить, они быстрее запускаются и приходят в работоспособное состояние.

Для работы с Docker можно использовать CLI или приложение Docker Desktop.

Документо-ориентированные базы данных на примере MongoDB {#документо-ориентированные-базы-данных-на-примере-MongoDB}

Установка и настройка MongoDB

Начало работы на общедоступном компьютере

 При выполнении практических заданий на общедоступном компьютере используйте предустановленное ПО для виртуализации (например, Oracle VM VirtualBox). И выполняйте

эксперименты в развернутой виртуальной машине. Рекомендуется использовать ОС **Ubuntu 2404 LTS** или **Ubuntu 22.04.5 LTS**.

🔗 Официальная [документация](#) Oracle VM VirtualBox

🔗 Подробнее про [виртуализацию](#) на РЕД ОС

Вариант 1: Официальный дистрибутив MongoDB (требуется VPN)

1. Загрузить дистрибутив MongoDB Community Edition с официального сайта [mongodb.com](https://www.mongodb.com).
2. Установить MongoDB на операционную систему (Windows, Linux или [MacOS](#)).
3. Запустить MongoDB-сервер и подключиться к нему с помощью клиентского приложения ([MongoDB Compass](#)) или командной строки `mongo` [sh](#). Определить где вам комфортнее работать.

Вариант 2: Контейнер с MongoDB в Docker

Рекомендуется выполнять лабораторную работу при помощи Docker-контейнера. Это не требует скачивания дистрибутива MongoDB с официального сайта.

Установить Docker с официального сайта [docker.com](https://www.docker.com) (если ссылка не работает, можно попробовать эту: [Get Docker](#)).

Можно использовать как CLI, так и десктопное приложение.

Docker CLI (на примере Ubuntu 2404 LTS)

Скачать скрипт, который позволяет автоматически собрать и развернуть все зависимости:

```
curl -fsSL https://get.docker.com -o get-docker.sh
```

Сделать его исполняемым:

```
sudo chmod +x get-docker.sh
```

Запустить:

```
sudo sh ./get-docker.sh
```

После установки убедитесь, что Docker работает, проверив его версию в терминале:

```
docker --version
```

Загрузить образ Mongo из Docker Hub:

```
docker pull mongo
```

Docker загрузит официальную версию Mongo.

После завершения загрузки можно проверить, что образ был успешно скачан:

```
docker images
```

Запустить контейнер Mongo:

```
docker run --name mongo-lab -d mongo
```

Эта команда создает контейнер с именем `mongo-lab` и запускает его в фоновом режиме. Проверить статус контейнера можно с помощью:

```
docker ps
```

Подключиться к работающему контейнеру и запустить `mongo shell` (интерфейс командной строки):

```
docker exec -it mongo-lab mongosh
```

Оказавшись внутри `mongo shell`, возможно вводить команды. Пример успешного подключения:

```
test>
```

Для выхода можно воспользоваться комбинацией `ctrl+D`.

Docker CLI в VS Code

Установить [расширение](#) в VS Code.

The screenshot shows the Docker Extension Pack for Visual Studio Code in the VS Code marketplace. The header includes the Docker logo, the name 'Docker', the publisher 'Microsoft', a link to 'microsoft.com', a download count of '48 085 986', and a star rating of '5 (106)'. Below this is the tagline 'Makes it easy to create, manage, and debug containerized applications.' and buttons for 'Отключить' (Disable), 'Удалить' (Uninstall), and a checkbox for 'Автоматическое обновление' (Automatic updates). The main section is titled 'Docker Extension Pack for Visual Studio Code' and describes its purpose: 'The Docker Extension Pack makes it easy to build, manage, and deploy containerized applications from Visual Studio Code.' It also mentions that the pack currently contains one extension, 'Container Tools'. On the right side, there is a 'Установка' (Installation) section showing the extension's identifier 'ms-azuretools.vscode-docker', version '2.0.0', and a size of '141.35 KB'. Below this is a 'Marketplace' section showing the extension was published '9 г./лет назад' (9 years ago) and the last update was '5 мес. назад' (5 months ago).

Перейти во вкладку Containers, выбрать образ Mongo для разворачивания контейнера (Создать контейнер разработки...).

Выберите действие для создания контейнера разработки

Открыть папку в контейнере...

Открыть рабочую область в контейнере...

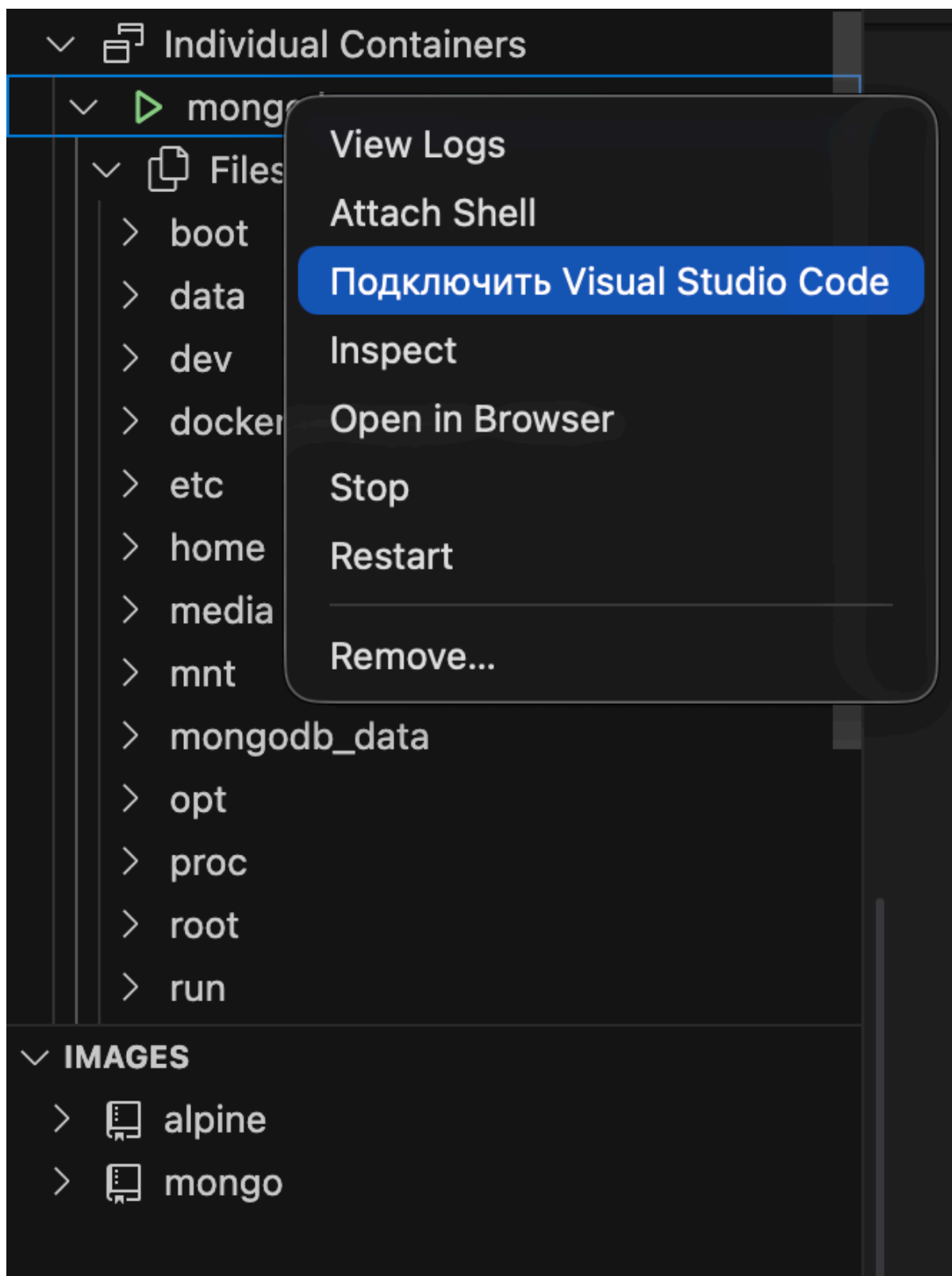
Создать контейнер разработки...

Клонировать репозиторий в томе контейнера...

Клонировать запрос на вытягивание GitHub в томе контейнера...

Попробовать образец...

Выбрать пункт “Подключить Visual Studio Code” в контекстном меню, вызванном на контейнере.

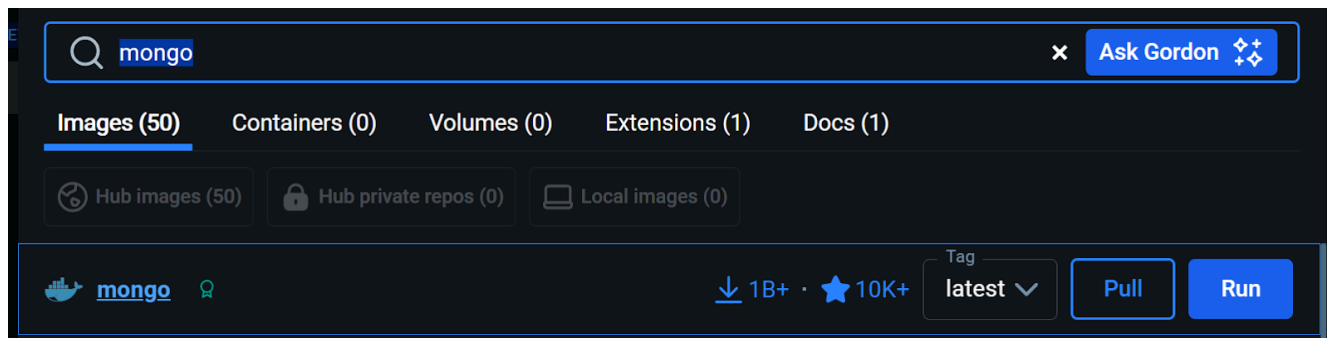


**

Начать работу в терминале.

Docker Desktop

Если используете установленное приложение Docker Desktop, в строке Search (Ctrl+K) ввести "mongo" и выбрать образ из списка:



Pull позволит загрузить образ, Run — дополнительно создать контейнер и запустить его.

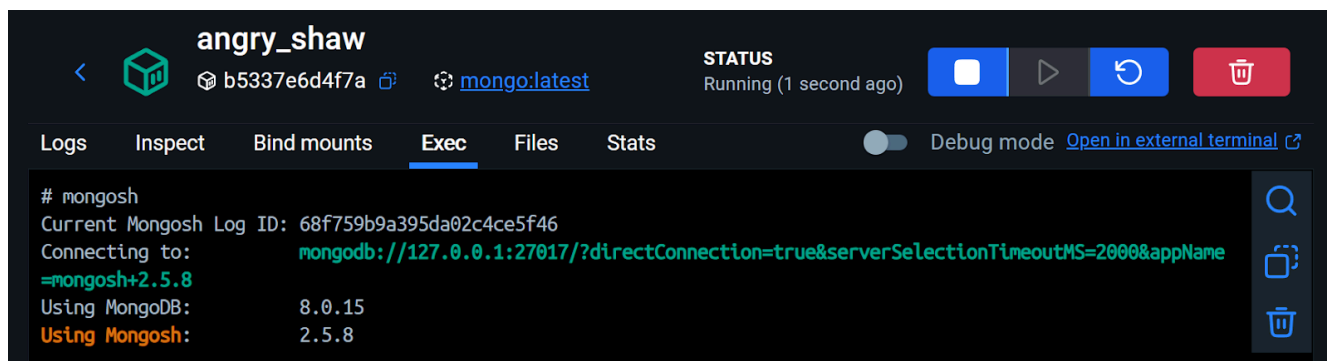
Прогресс загрузки образа можно наблюдать во вкладке Images.

Во вкладке Containers находятся созданные контейнеры. Работать в запущенном контейнере можно во вкладке Exec.

Для запуска Mongo Shell необходимо выполнить команду:

```
mongosh
```

Пример успешного подключения:



Основные операции в MongoDB

Предлагается работать с базой данных `studentsDB`, которую необходимо создать и заполнить самостоятельно, и коллекцией `students`, которая содержит информацию о студентах (ФИО, возраст, факультет, список дисциплин).

Создание базы данных и коллекции

```
use studentsDB;  
db.createCollection("students");
```

Вставка данных в коллекцию

```
db.students.insertOne({  
  "name": "Иван Иванов",
```

```
"age": 21,  
"faculty": "Информатика",  
"courses": ["Математика", "Программирование", "Базы данных"]  
});
```

Выборка и фильтрация документов

Получение всех документов

```
db.students.find();
```

Фильтрация по возрасту

```
db.students.find({ "age": { $gt: 20 } });
```

Выборка только определённых полей

```
db.students.find({}, { "name": 1, "faculty": 1 });
```

Обновление и удаление документов

Обновление значения поля

```
db.students.updateOne({ "name": "Иван Иванов" }, { $set: { "age": 22 } });
```

Удаление документа

```
db.students.deleteOne({ "name": "Иван Иванов" });
```

 [Шпаргалка](#) по командам MongoDB.

 Более подробная [шпаргалка](#) по CRUD командам MongoDB.

Дополнительные запросы и операции

В этом разделе рассматриваются более продвинутые запросы и операции в MongoDB, которые могут понадобиться для работы с данными. Понимание этих тем позволит вам оптимизировать запросы, выполнять сложные выборки и эффективно управлять структурой данных.

Поиск по вложенным структурам

В MongoDB документы могут содержать вложенные структуры (например, массивы или объекты), что позволяет моделировать сложные данные. Поиск по вложенным структурам требует использования специального синтаксиса и операторов MongoDB, таких как `dot notation` для указания вложенных полей.

Поиск по вложенному полю объекта

Предположим, что у нас есть коллекция `students`, где документы хранят информацию о студенте, включая его контакты.

```
{
  "name": "Иван Иванов",
  "age": 21,
  "faculty": "Информатика",
  "contacts": {
    "email": "ivanov@example.com",
    "phone": "123-456-7890"
  }
}
```

Чтобы найти студента по его email, нужно использовать следующий запрос.

```
db.students.find({ "contacts.email": "ivanov@example.com" });
```

Здесь используется dot notation (`contacts.email`), чтобы указать MongoDB, что необходимо искать значение `email` во вложенном объекте `contacts`.

Поиск по элементам массива

Рассмотрим другой пример, где в документах хранится список курсов, которые проходит студент.

```
{
  "name": "Петр Петров",
  "age": 22,
  "faculty": "Математика",
  "courses": [
    { "name": "Алгебра", "credits": 5 },
    { "name": "Геометрия", "credits": 3 }
  ]
}
```

Чтобы найти всех студентов, которые проходят курс с названием «Алгебра», можно воспользоваться следующим запросом.

```
db.students.find({ "courses.name": "Алгебра" });
```

Такой запрос отыщет все документы, где массив `courses` содержит объект с полем `name`, равным «Алгебра».

Поиск по нескольким условиям во вложенных структурах

Если нужно искать документы по нескольким условиям во вложенных структурах, можно воспользоваться оператором `$elemMatch`. Предположим, что нам нужно найти всех студентов, которые изучают курс «Геометрия» и получают за него более 4 кредитов.

```
db.students.find({
  "courses": {
    $elemMatch: { "name": "Геометрия", "credits": { $gt: 4 } }
  }
});
```

Этот запрос проверяет наличие элемента в массиве `courses`, который соответствует обоим условиям одновременно.

Агрегация данных

Агрегация данных в MongoDB используется для выполнения операций, аналогичных `GROUP BY` в реляционных базах данных. Агрегация позволяет выполнять сложные выборки, группировать данные, применять фильтрацию и сортировку. Основной командой для агрегации является `aggregate`.

Составные части агрегатных запросов:

- **\$match**: фильтрация документов по условиям (аналог `WHERE`).
- **\$group**: группировка данных на основе заданного поля и применение агрегатных функций (`sum`, `avg`, `max`, `min`).
- **\$project**: создание нового представления данных, отображение или скрывание полей.
- **\$sort**: сортировка по указанным полям.
- **\$limit** и **\$skip**: ограничение количества возвращаемых документов.

Пример агрегации данных

Простая агрегация с подсчетом количества студентов на каждом факультете.

```
db.students.aggregate([
  { $group: { _id: "$faculty", studentCount: { $sum: 1 } } }
]);
```

Этот запрос группирует документы по полю `faculty` и подсчитывает количество студентов на каждом факультете с помощью `$sum: 1`.

```
db.students.find({ "contacts.email": "ivanov@example.com" });
```

Здесь используется `dot notation` (`contacts.email`), чтобы указать MongoDB, что необходимо искать значение `email` во вложенном объекте `contacts`.

Агрегация с фильтрацией и группировкой

Предположим, что мы хотим подсчитать количество студентов на каждом факультете, которые старше 20 лет.

```
db.students.aggregate([
  { $match: { "age": { $gt: 20 } } },
  { $group: { _id: "$faculty", studentCount: { $sum: 1 } } }
]);
```

На первом этапе (`$match`) происходит фильтрация документов, затем группировка по полю `faculty` и подсчёт количества студентов.

Создание нового представления данных с `project`

Чтобы создать новое представление данных и исключить некоторые поля, можно использовать `$project`. Например, оставить только имя студента и его возраст.

```
db.students.aggregate([
  { $project: { name: 1, age: 1, _id: 0 } }
]);
```

Этот запрос выводит только поля `name` и `age` для каждого студента, исключая поле `_id`.

Вложенные этапы агрегации и сортировка

Допустим, нужно найти студентов, которые изучают более 2 курсов, и отсортировать их по количеству курсов.

```
db.students.aggregate([
  { $project: { name: 1, courseCount: { $size: { $ifNull: ["$courses", []] } } } },
  { $match: { courseCount: { $gt: 1 } } },
  { $sort: { courseCount: -1 } }
]);
```

В этом примере сначала создается новое поле `courseCount`, которое вычисляется на основе размера массива `courses`, затем происходит фильтрация и сортировка в порядке убывания.

Применение индексов для ускорения запросов

Индексы в MongoDB помогают ускорять выполнение запросов, особенно когда нужно искать данные по одному или нескольким полям. Индексы создают дополнительную структуру данных, которая позволяет быстро находить документы без необходимости проходить через все записи коллекции.

Типы индексов:

- **Одиночные индексы (Single Field Index):** индекс по одному полю.
- **Составные индексы (Compound Index):** индекс по нескольким полям.
- **Текстовые индексы (Text Index):** индекс для текстового поиска.
- **Уникальные индексы (Unique Index):** гарантируют уникальность значений поля.

Создание одиночного индекса

```
db.students.createIndex({ "name": 1 });
```

Индекс создается по полю `name` в порядке возрастания (1). Теперь запросы по имени студента будут выполняться быстрее.

Команда `getIndexes()` позволяет просмотреть все индексы в коллекции.

Создание составного индекса

```
db.students.createIndex({ "faculty": 1, "age": -1 });
```

Индекс создается по полям `faculty` (по возрастанию) и `age` (по убыванию). Такой индекс будет полезен при выполнении запросов, которые фильтруют данные по факультету и сортируют их по

возрасту.

Удаление индексов

```
db.students.dropIndex("name_1");
```

Удаляет индекс, созданный по полю `name`.

Пример применения индексов

Создание индекса для ускорения поиска по курсам.

```
db.students.createIndex({ "courses.name": 1 });
```

Теперь все запросы, которые ищут документы по курсам, будут выполняться быстрее.

```
db.students.find({ "courses.name": "Алгебра" });
```

Индексы — это мощный инструмент для оптимизации запросов в MongoDB. Однако важно помнить, что использование индексов увеличивает время вставки и обновления данных, а также объем занимаемой памяти. Поэтому рекомендуется создавать индексы только на тех полях, по которым часто выполняются запросы или сортировка.

 [Операторы](#) запроса сравнения.

Самостоятельная работа {#самостоятельная-работа}

Почему актуальны документо-ориентированные базы данных?

Документо-ориентированные базы данных, такие как MongoDB, стали популярными благодаря своей способности гибко управлять неструктурированными и полуструктурированными данными. В отличие от традиционных реляционных баз данных, такие решения позволяют работать с разнородными структурами, что делает их идеальными для хранения и анализа данных в реальном времени, для создания систем, которые требуют гибкой схемы (например, веб-приложений, систем управления контентом, аналитических платформ и логов).

Цель

Изучить основы работы с документо-ориентированными базами данных на примере MongoDB, освоить базовые и продвинутые операции с данными, включая CRUD, индексацию и агрегацию.

Инструкция

1. Определите вариант задания самостоятельно ([возможные варианты](#) могут помочь с выбором). Каждое задание представляет реальный сценарий работы с базой данных и направлено на изучение ключевых аспектов работы с MongoDB. Задания подразумевают погружение на 4-6 часов в зависимости от уровня подготовки.
2. Для выбранного сценария разработайте или подберите в открытых источниках (JSON-подобные документы) коллекцию с различными типами документов, моделирующими

предметную область (например, книги, товары, сотрудники и т.д.). Для загрузки файла в CLI воспользуйтесь [mongoimport](#).

3. Реализуйте операции создания, чтения, обновления и удаления (CRUD) документов в коллекции.
4. Выполните запросы для поиска данных с различными условиями (по отдельным полям, по вложенным структурам).
5. Создайте индексы для ускорения поиска и анализа данных (особенно актуально для коллекций с большим количеством документов).
6. Выполните агрегатные запросы для получения сводной информации, статистики и сложной аналитики (например, подсчет количества документов по категориям, группировка, сортировка).

Каждое задание моделирует реальный сценарий, с которым могут столкнуться специалисты по работе с данными или разработчики.

Возможные варианты {#возможные-варианты}

Вариант 1: Аналитика логов веб-приложения

Создайте коллекцию для хранения логов веб-приложения, содержащую поля: время события, уровень логирования (INFO, WARNING, ERROR), сообщение, идентификатор пользователя, URL запроса.

1. Создайте коллекцию и добавьте лог-записи с различными уровнями логирования.
2. Выполните запросы для поиска ошибок за последний месяц.
3. Реализуйте индексацию по времени и идентификатору пользователя.
4. Сгруппируйте логи по URL и определите на какие страницы было больше всего запросов.
5. Постройте отчет по количеству ошибок на пользователя и выявите пользователей с наибольшим числом ошибок.

Цель: освоить навыки работы с логами и анализа ошибок на примере MongoDB.

Вариант 2: Система управления контентом (CMS)

Создайте коллекцию для хранения данных о статьях, содержащую: заголовок, автор, дата публикации, теги, текст статьи, количество просмотров.

1. Создайте коллекцию с несколькими статьями, используя вложенные структуры для тегов и авторов.
2. Выполните запросы для поиска статей по тегам и дате публикации.
3. Добавьте индексы по тегам и количеству просмотров.
4. Агрегируйте данные по авторам и определите, чьи статьи получили наибольшее количество просмотров.
5. Найдите топ-3 популярных тега, которые встречаются чаще всего.

Цель: научиться работать с текстовыми данными и анализировать популярность контента.

Вариант 3: Моделирование профилей пользователей

Создайте коллекцию, представляющую данные о пользователях: имя, возраст, пол, город, хобби, история заказов (вложенная структура).

1. Добавьте в коллекцию данные о нескольких пользователях с разнообразными профилями и историей заказов.
2. Реализуйте запросы для поиска пользователей по городу и возрасту.
3. Сгруппируйте пользователей по хобби и найдите хобби, которые наиболее популярны.
4. Примените индексы для повышения скорости поиска по городу и хобби.
5. Выполните агрегацию для определения среднего возраста пользователей по каждому городу.

Цель: разработать структуру и запросы для эффективного анализа профилей пользователей и их сегментации.

Вариант 4: Управление проектами и задачами

Создайте коллекцию для хранения информации о проектах: название проекта, дата начала, дата завершения, статус, задачи (вложенная структура с описанием, исполнителем, приоритетом).

1. Создайте коллекцию проектов с вложенными структурами задач.
2. Выполните запросы для поиска задач с высоким приоритетом.
3. Сгруппируйте задачи по статусу и подсчитайте количество задач в каждом статусе.
4. Реализуйте индексацию по дате начала и статусу задач.
5. Агрегируйте данные для получения статистики завершенных и незавершенных задач по каждому проекту.

Цель: разработать систему управления задачами и проектами на примере MongoDB.

Вариант 5: Учёт продажи товаров в магазине

Создайте коллекцию для учета продаж и товаров: название товара, категория, цена, количество на складе, заказы (вложенная структура с датой заказа, количеством, суммой).

1. Создайте коллекцию с данными о товарах и их продажах.
2. Реализуйте запросы для поиска товаров по категории и цене.
3. Сгруппируйте товары по категориям и определите, какие категории продаются лучше всего.
4. Выполните индексацию по количеству и цене товаров.
5. Агрегируйте данные для определения средней суммы заказа и общего дохода за период.

Цель: научиться работать с товарными данными и аналитикой продаж на примере MongoDB.

Вариант 6: Исследование пингвинов

Используя [готовый датасет](#), создайте коллекцию для хранения данных о пингвинах, содержащую поля: вид пингвина, длина клюва (mm), глубина клюва (mm), длина лап (mm), масса тела (g), пол пингвина, остров обитания.

1. Загрузите данные из датасета в созданную коллекцию.
2. Выполните запросы для поиска пингвинов с длиной клюва более 50 мм.
3. Реализуйте индексацию по виду пингвина и длине клюва.
4. Сгруппируйте данные по виду пингвина и определите средние значения длины клюва и массы тела для каждого вида.
5. Постройте отчет по количеству пингвинов каждого вида на каждом острове и выявите остров с наибольшим количеством пингвинов.

Цель: освоить навыки работы с данными на примере MongoDB.

Проверка выполненного отчета {#проверка-выполненного-отчета}

Общие требования

Отчет по лабораторной работе должен демонстрировать понимание и практическое применение концепций, изученных в ходе занятия. Он должен быть структурированным, содержать теоретическое обоснование, пошаговое описание выполнения работы, а также критический анализ полученных результатов.

Когда работа выполнена

Проверьте себя, чтобы убедиться, что в отчете описан результат выполнения с пояснениями и/или приложены корректные ссылки на необходимые доступные файлы:

- ☐ В отчете описан каждый этап с пояснениями и приложены скриншоты (либо программный код в формате текста) выполнения работы.

Дополнительно убедитесь:

- ☐ Отчет сохранен с корректным названием (указано ФИО).
- ☐ Доступ к отчету открыт для всех, у кого есть ссылка, и включена возможность комментирования.
- ☐ Финальная версия отчета отправлена преподавателю на проверку.

Дополнительные материалы {#дополнительные-материалы}

1. [Николаев, Е.И. Основы NoSQL СУБД: Учебное пособие \(лабораторный практикум\) для студентов специальности 09.03.02 Информационные системы и технологии. Ставрополь, 2016.](#)
2. [Название видео: Mongo DB ПОЛНЫЙ КУРС | Автор: Ulbi TV | Дата публикации: Jan 2, 2021.](#)
3. [Код учебного курса “MongoDB” на YouTube-канале webDev.](#)
4. [От А до Я: шпаргалка по MongoDB.](#)